Chapter 1: Introduction and Overview

Database system == client/server model == database nodes are the servers + applications that use the database are the clients.

Database Architecture

Database architecture has the following components:

1. **Transport subsystem**

   i. Sending and receiving data between the client and the database.

   ii. Communication between the other nodes of the DB cluster.

2. **Query processor**

   i. Parse, validate, and optimize queries.

3. **Execution engine**

   i. Execute the query. This could be local or remote execution. (remote execution is when the query is executed on a different node)

4. **Storage engine**

   i. Helps the execution engine to read and write data from the storage.

   ii. Responsible for the storage layout and the storage medium.

Lifecycle of a query

Client request -> Transport subsystem -> Query processor -> Execution Engine -> Storage engine -> Execution engine -> Transport subsystem -> Client response

Storage Engine

Since part 1 of the book is about storage engines, we look at the storage engine in more detail.

It has several components:

1. **Transaction Manager**

   i. Responsible for scheduling transactions.

   ii. Makes sure queries do not cause inconsistencies in the database.

2. **Lock Manager**

   i. Responsible for concurrency control, thus preventing violation of data integrity.

3. **Access Methods (storage structures)**

   i. Facilitates access to data + organizing data on disk.

   ii. Examples could be B trees, LSM trees, etc

4. **Buffer Manager**

   i. Responsible for caching data pages in memory.

5. **Recovery Manager**

   i. Responsible for recovering the database state in the event of failures.

Dividing databases into categories

```
graph LR

%% Top Node

databases(Categories of Databases)
use_case(Use case)
storage_medium(Storage Medium)
storage_layout(Storage Layout)

%% Use case
OLTP("Online Transaction Processing (OLTP)")
OLAP("Online Analytical Processing (OLAP)")
HTAP("Hybrid Transactional/Analytical Processing (HTAP)")

%% Storage medium
Disk(Disk)
InMemory(In-memory)

%% Storage layout
Row(Row-Oriented)
Column(Column-Oriented)

databases --> use_case & storage_medium & storage_layout
use_case --> OLTP & OLAP & HTAP
storage_medium --> Disk & InMemory
storage_layout --> Row & Column
```

OLTP vs OLAP vs HTAP Databases

OLTP == optimized for transactional workloads. Often used in user-facing requests and transactions.

OLAP == optimized for analytical workloads. Often used for analytics and data warehousing.

HTAP == optimized for both transactional and analytical workloads.

> Why not use HTAP?
> HTAP is a compromise between OLTP and OLAP. It has both things but it's not the best at either of

> them. It is a compromise.

Disk vs In-memory Databases

Type of database	Use memory for	Use disk for
In-memory	storing data	recovery and logging
Disk	caching + temporary storage	storing data

Idea here is that accessing RAM is much faster than accessing disk, but RAM is volatile.

NVM storage (non-volatile memory) is a new technology that is being used to store data in memory. It is non-volatile, so it is not lost when the power is turned off.

Durability in in-memory databases

Since in-memory databases are volatile, they need to be made durable. This is done by maintaining backups on disk. But how?

Checkpointing

It has 2 components:

1. **Sequential log file:** Before any operation is called successful, the database must ensure that the data is written to a sequential log file. This is done by using a technique called *write-ahead logging*, which is a topic for a future chapter.
2. **Backup copy:** Another thing we store on disk is a *backup copy* of the database. The "checkpointer" is a component of recovery manager that is responsible for maintaining a backup copy of the database in an almost up-to-date state. (Source / Good read: https://pages.cs.wisc.edu/~vijayan/ra/papers/checkpointing-memory-resident-db.pdf)

How do these 2 components work together?

Log entries are applied to the backup copy in batches **asynchronously**. When the batch is applied, the log entries are removed from the log file, and the backup copy holds a database snapshot for a specific point in time. This process is called *checkpointing* and ensures that the database is durable.

Why do we need to discard the log entries after they are applied to the backup copy?

Because the log file can grow very large, and we don't want that as that would slow down the recovery process.

A small example for visualization

NOTE: This is ONLY FOR VISUALIZATION. This does NOT reflect the actual implementation and intricacies of checkpointing.

Let's say current state of the database is

```
id=1, name=akshat
id=2, name=mohit
```

Let's say current state of the log file has entries for id=1 and id=2.

```
t=20 October 2023, 10:00:00: `id=1, name=akshat`
t=20 October 2023, 10:00:01: `id=2, name=mohit`
```

Let's say the current state of backup copy has not been updated since the last checkpoint, and contains the following entries:

```
id=1, name=akshat
```

Let's say we crash the system before the checkpointer has had a chance to apply the log entries to the backup copy. The recovery manager has all the data needed to recover the database to the actual state by reading log entries + backup copy.

The log entries can be discarded after the backup copy is updated because the information is redundant. The backup copy already has the data, so we don't need the log entries anymore till now. This is done by the checkpointer.

TLDR: Checkpointing is essentially the following:

```
expected_state_of_db = apply_log_entries_to_backup_copy(backup_copy, log_entries)
```

Row vs Column Oriented Databases

Row oriented databases store data in row-manner on the disk.

Column oriented databases store data in column-manner on the disk.

Very basic example could look like the following.

Row oriented database:

```
id=1, name=akshat, age=20
id=2, name=mohit, age=21
```

Column oriented database:

```
id=1, 2
name=akshat, mohit
```

```
age=20, 21
```

Data in disk is stored block-wise.

> Block is a minimal unit of data that can be read from or written to disk.

Queries like `SELECT name FROM users` would be faster in column oriented databases because we don't have to read the entire row, we just have to read the name column. This is because the different values for the column are stored together in the disk, hence we have to read less number of blocks. (ofcourse not for just 2 teeny tiny records, but you get the idea!)

Queries like `SELECT * FROM users where id=1` would be faster in row oriented databases because we don't have to read the entire column, we just have to read the row. This is because the different values for the row are stored together in the disk, hence we have to read less number of blocks.

TLDR: Row oriented stores improve spatial locality for rows, column oriented stores improve spatial locality for columns.

> Column oriented databases are used for OLAP workloads because the use-cases involve aggregations and analytical computations on lesser columns. Row oriented databases are used for OLTP workloads because they are optimized for transactional queries.

Another interesting difference

Reading multiple values for same column in one run has some other interesting aspects in column oriented databases:

1. This improves cache utilization.
2. This has a better compression ratio.
3. This improves vectorization.

Wide column stores

Examples: BigTable, HBase

Data is represented as a multidimensional map. Columns are grouped into column families. Inside each column family, the data is stored row-wise.

I don't fully understand the practical use-cases of this. I wasn't able to get good intuition on this from the book, so I'll skip this for now.

Data Files and Index Files

This was the most confusing part of the chapter for me. I'll try to explain it as best as I can. Please correct me if I'm wrong.

This is about how data is organized on disk. 2 types of files are used:

1. Data files: These contain the actual data.
2. Index files: These contain the metadata for records of data files, to help us find the data we want efficiently.

Files are partitioned into pages. 1 page == one or many disk "blocks".

Data Files

Data files can be organized in 3 ways:

1. Heap files: Data is stored in an unordered manner, mostly in a write-order.
2. Hash files: Bucketed approach of storing data. Hash of the key is used to determine the bucket in which the data is stored.
3. Index organized tables (IOT): Data is stored in the index itself.

Depending on the type of data file, the index file can have different contents (called "data entries"):

1. Heap files: Index file contains the file offsets (row locators).
2. Hash files: Index file contains the bucket IDs.
3. Index organized tables (IOT): Index file contains the primary key.

Index Files

An index is a data structure that helps us find the data we want efficiently.

Types of index: Index on data file (primary file) is called **primary index** (also called primary index if it is on a composite primary key). All other indexes are called **secondary**.

Clustered vs non-clustered index: If the data is stored in the same order as the index, it is called a clustered index. Otherwise, it is called an non-clustered index.

The above differences are explained quite confusingly in the book. I'll update this section if I find a better explanation. I'm hopeful that chapter 2 will clear things up, as it has a lot of examples around this area.

Primary index as an indirection

2 ways of accessing data:

1. Reference it directly (through file offset)

   i. Benefit: Faster, since less number of disk seeks are required. ii. Drawback: Pointers need to be updated when data is moved around / updated.

2. Reference it indirectly (through primary key index)

   i. Benefit: Less cost of pointer updates. ii. Drawback: Slower, since more number of disk seeks are required for reading.

Buffering, Immutability, and Ordering

Storage structures have 3 variables:

1. Buffering: Whether the data is buffered in memory before being written to disk.
2. Immutability: Whether the data is updated in-place (mutable) or it's append-only (immutable).
3. Ordering: Whether the data is stored in the key order or not.

Pending Doubts

1. Is the "Lifecycle of a query" part above correct? (Especially in the 2nd half of the cycle where response is to be returned)

[Update] Answer: Yes, it is correct.

Drop me an email if you would like to answer any of the doubts, or discuss: Let's talk!

Things to Read

1. NVM storage - current state, timelines, benchmarks, etc
2. Vector instructions / Vectorization in column oriented databases
3. Wide column stores - use cases, practical examples, etc

Reading group discussion

This section contains anything worth mentioning that came up as part of the weekly reading group discussion.

1. What is a possible migration plan from Database-A to Database-B ?

   i. A very interesting story of a migration from Couchbase to Aerospike: Large Scale NoSQL Database Migration Under Fire

   ii. Online migrations at scale

2. If you wonder about the rationale behind design decision of DBs implementing their own data cache (buffer pools), and forgoing pagecache of OS and using direct I/O on underlying filesystem, here's a good paper by Andy Pavlo: https://db.cs.cmu.edu/papers/2022/cidr2022-p13-crotty.pdf

3. Request lifecycle

An interesting example was mentioned by someone on my doubt on request lifecycle.

Once you arrive at the execution engine, it's not so linear.

As an example, take `SELECT * FROM foo WHERE bar IN (SELECT bar FROM boop)`.

The execution engine will end up with a plan like:

```
1. Read table boop's bar field into List A.
2. Sort List A (to make step 4's search faster).
3. Read each row from table foo.
4. Emit each all the fields from foo into the result set if foo.bar in List A.
```

So we would have the execution engine reading from the storage engine in (1), doing some internal processing, then going back out to the storage engine to read table foo's rows in step (3). There's an interaction between the two of them as execution proceeds. As we increase complexity of queries, introduce indexes, etc etc etc, it's very easily going to get complicated. We might even have several storage engines involved in a single query (eg, indexes and primary data in different engines).

Step 4 might emit the rows directly to the client, or it might buffer them before returning. If there's a sort, group etc. involved it might *have* to buffer the results.

Chapter 2: B-Tree Basics

B-Trees are one of the most popular storage structures used in databases.

Binary Search Trees (BST)

A binary search tree is a sorted, in-memory data structure that is used for efficient key-value lookups.

BST invariant: For any node, all the keys in its left subtree are less than the node's key, and all the keys in its right subtree are greater than or equal to the node's key.

Tree balancing: If the tree is not balanced, the height of the tree can be as high as the number of nodes in the tree. This can lead to O(n) lookup time.

Balanced tree == height of the tree is O(log n), where n is the number of nodes in the tree + difference in height of left and right subtrees is at most 1.

We balance the tree by rotating the nodes. The rotation is done in such a way that the BST invariant is maintained. For example:

```
Before balancing:
        3
      /
     2 (pivot!!)
    /
   1

After balancing:
        2
      /   \
     1     3
```

Trees for Disk-Based Storage

(Fanout == branching factor == number of children a node can have)

BSTs are not suitable for disk-based storage because of the following reasons:

1. Locality: New element might not be written to the same page as its parent, leading to more disk seeks
2. Low fanout == more height == more disk seeks required to find the element

Storage types

Before we discuss better structures suitable for disk-based storage, let's discuss the different types of storage available.

Hard disk drives (HDDs): HDDs are mechanical devices that have a spinning disk with a read/write head. The read/write head moves to the location of the data to be read/written. HDDs are slow because of the mechanical movement involved to position the read/write head to the correct location. The reading/writing itself is fast once the head is positioned correctly.

Smallest unit of data that can be read/written to an HDD is a sector (typically ranges from 512 bytes to 4 KB).

Solid state drives (SSDs):

SSDs are faster than HDDs because they don't have any moving parts.

SSD structure:

1. SSDs have one or more dies.
2. Each die has multiple planes.
3. Each plane has multiple blocks.
4. Each block has multiple pages.
5. Each page has multiple arrays.
6. Each array has multiple strings.
7. Each string has multiple memory cells.

🦁 In SSDs, the data is read/written at the page level, but is erased at the block level.

✨ **Side learning:** But why? This is because of the physical properties of quantum tunneling. Long story short, a significant amount of voltage is required to erase data, and it is difficult to target that voltage at a more granular level without negatively affecting the surrounding cells. Hence, the smallest unit of data that can be erased is a block.

Source:

1. https://qr.ae/pKHTdl
2. https://www.techtarget.com/searchstorage/definition/solid-state-storage-SSS-garbage-collection

On-disk data structures: B-Trees

B-Trees are based on binary search trees, but have higher fanout, and hence lesser height.

Binary Tree == 1 key + 2 child pointers per node. 2-3 Tree == 2 keys + 3 child pointers per node.

Property: All keys inside a B-Tree node are sorted. To locate a key within a node, we can use binary search.

Occupancy of a node == number of keys in a node in comparison to the maximum number of keys that can be stored in a node.

B-Tree characteristics:

1. Higher fanout leads to amortized cost of keeping the tree balanced.
2. Reduced number of seeks by storing keys and child pointers in a single block or consecutive blocks.
3. Balancing operations happen when the nodes are either full or nearly empty.

Separator Keys

Separator Keys == Index entries == Divider cells

Say "p" denotes a child pointer, and "k" denotes a key. Then, a separator key is of the form:

```
p1 k1 p2 k2 p3
```

Here:

1. p1 points to the child node with keys < k1
2. p2 points to the child node with keys >= k1 and <= k2
3. p3 points to the child node with keys > k2

Sibling pointers

Sibling pointers == Next and previous pointers == Link to the next and previous node at the same level

B-Trees have sibling pointers on the leaf nodes. This allows us to traverse the leaf nodes in a linked list fashion, and helping range queries (as we don't have to go back to the parent to find the next node).

B-Tree Lookup Complexity

log M, where M is the total number of nodes in the tree.

B-Tree Node Splitting

When a node is full, we split it into two nodes. The middle key is moved to the parent node, and the remaining keys are distributed between the two nodes.

Say we have a maximum occupancy of 3 keys per node.

Parent node: 18 _ _ Left pointer of 18 points to a node having: 10 13 15

If we insert 11, we will have to split the child node since we cannot have "10 11 13 15" in a single node.

So, we split "10 11 13 15" into "10 11" and "13 15". The middle key "13" is moved to the parent node (while keeping the parent node's keys sorted), and the remaining keys are distributed between the two nodes.

So, final state of the tree will be: Parent node: 13 18 _ Left pointer of 13 points to a node having: 10 11 _ Pointer between 13 and 18 points to a node having: 13 15 _

You can refer to diagram on page 40 of the book for a visual representation.

B-Tree Node Merging

Deletions are done by removing the key and value from the leaf node. If the neighbouring nodes have too few values, the sibling nodes are merged. This is called underflow.

Example:

```
Parent node: 11 20 _
Pointer between 11 and 20 points to a node having: 15 16 _
Pointer between 20 and _ points to a node having: 25 30 _

If we delete 16, we will have to merge the two nodes since the occupancy of the
two nodes will be 15 _ _ and 25 30 _ respectively.

For merging, we move elements from the right sibling node to the left sibling
node, and update the parent node accordingly.

So, final state of the tree after merging will be:
Parent node: 11 _ _
Pointer between 11 and _ points to a node having: 15 25 30.
```

Pending Doubts

1. Page 35 says "B-Trees are a page organization technique (i.e., they are used to organize and navigate fixed-size pages), we often use terms node and page interchangeably". What does it mean? How are nodes equivalent to pages?

[Update] Answer: We understand this better in chapter 3.

Other answers I got from other members of the reading group: As far as implementations go, databases talk in terms of pages instead of blocks. For instance, in SQLite's BTree implementation, they exclusively talk in terms of pages. The size of the page can be between 512 bytes and 65536 bytes, and each page contains pointers to other pages.

In the case of Postgres, things become a bit more complicated. For instance, they say they split a heap or index file in blocks. But a block can be unformatted, whereas a page is a well formed slotted page. All the access method implementations (including BTree) work with pages.

2. SSD has page inside blocks. But in chapter 1, we saw that database had 1 page = multiple blocks? Any historic context on why we have this discrepancy in naming? Any intuitive explanation for this?

[Update] Answer: It's just confusing, but it's what it is.

Other answers I got from other members of the reading group: It seems that everyone uses the terms differently depending on context. For example, there is the context of the physical disk, of the interface the disk uses to communicate (e.g. NVMe, NTFS, etc.), the file system, and databases. It doesn't help that every vendor at every layer there often uses a different or varied term too.

Databases mostly seem to use the term "page" to refer to the minimum number of bytes they read or write. Whereas "block" seems to more refer to either the file system, disk protocol (NVMe, etc.), or physical disk minimums. And "page" is always a multiple of the block size so that you aren't reading/writing partial blocks which incurs additional IO.

Various sites to increase the confusion even further:

- https://superuser.com/questions/1110355/what-is-the-difference-between-a-disk-block-and-a-sector

- https://www.reddit.com/r/sysadmin/comments/37a8xi/eli5_the_differences_between_a_chunk_page_block/
- https://dba.stackexchange.com/a/241042
- https://wiki.archlinux.org/title/Advanced_Format

Drop me an email if you would like to answer any of the doubts, or discuss: Let's talk!

Things to Read

NA at the moment.

Reading group discussion

This section contains anything worth mentioning that came up as part of the weekly reading group discussion.

1. There is a good explanation of B-Trees in the Introduction to Algorithms book by Cormen that you may have lying around from school or work. It includes a nice explanation of how (spinning) disks work which is rare for an algorithms book.

2. Good explanation of Red-Black and 2-3 trees: https://www.cs.umd.edu/class/fall2019/cmsc420-0201/Lects/lect06-aa.pdf

3. Fly.io has a great blog, here's one from Ben Johnson about SQLite btrees: https://fly.io/blog/sqlite-internals-btree/

Chapter 3: File Formats

Data layout is much more important in disk than in memory. For a disk-based data structure to be efficient, we need to:

1. lay it out in a way that allows quick/efficient access to the data we need.
2. consider the specific characteristics of the underlying storage medium.
3. come up with binary data formats, and also ways to serialize and deserialize data.

Binary Encoding

To store data on disk, we need a format that is compact + easy to serialize and deserialize. We can use binary encoding for this purpose.

Primitive Types

Types like integer, data, and string can be represented in their raw binary forms. However, for multi-byte numeric values, we need to consider the byte order (endianness) of the system.

✨ **Side learning:** Why do we still have endianness issues in modern systems? There's no good answer to this, we've just failed to standardize on a single byte order:
https://softwareengineering.stackexchange.com/a/301184

2 types of endiannes:

1. Big-endian: Most significant byte first.
2. Little-endian: Least significant byte first.

Floating point numbers are represented in IEEE 754 standard. A 32 bit number is represented as:

1. 1 bit for sign
2. 8 bits for exponent
3. 23 bits for mantissa

Strings and Variable-Size Data

Variable-sized data types such as Strings and Arrays of fixed-size data can be serialized as a number (size) followed by the actual data.

For example, a string "Hello" can be serialized as:

```
5Hello
```

An alternative approach is to use a special character to mark the end of the string. For example, a string "Hello" can be serialized as:

```
Hello\0
```

Bit-Packed Data

Some data types are not byte-aligned, and it's a waste to use the whole byte to store them. For example, a boolean value can be represented in 1 bit. We can use bit-packing to store such data types.

1. Boolean: We can say that 1 is true and 0 is false.
2. Enums can be represented as a number, and we can use bit-packing to store them.
3. Flags: Flag is a combination of boolean values and enums. If we have 3 flags, we can use 3 bits, each representing the value of a flag.

General Principles

The file usually starts with a fixed-size header and may end with a fixed-size trailer. The rest of the file is split into pages. A very raw diagram of a file is as follows:

```
+-------------------+
| Header            |
+-------------------+
| Page 1            |
+-------------------+
| Page 2            |
+-------------------+
| ...               |
+-------------------+
```

```
| Page N            |
+-------------------+
| Trailer           |
+-------------------+
```

If we have a structure having a mix of fixed-size and variable-size fields, we can store the fixed-size fields in the structure itself,

Page Structure

The data and index files (that we studied in chapter 1) are partitioned into fixed-size units called pages (ranging from 4KB to 16KB).

Example of an on-disk B-Tree node Each B-Tree node occupies one or more pages linked together. So we interchangably use the terms page and node. A B-Tree node can be represented as follows:

```
-------------------------------------------------------------------
| p0 | k1 | v1 | p1 | k2 | v2 | ... | kn | vn | pn | Unused space |
-------------------------------------------------------------------
```

where:

- p = pointers to child pages
- k = keys
- v = values

Downsides of this approach:

1. Inserting a key-value pair in the middle of the node requires shifting all the subsequent key-value pairs.
2. Doesn't allow managing variable-size data efficiently.

Slotted Pages

Main problem with variable-size data is free space management, that is, reclaiming space from deleted records and reusing it for new records.

We can simply re-write the whole page whenever we need to update a record, but that would be inefficient and we would have to preserve record offsets as any out-of-page pointers might be referencing them. Hence, instead of this, we use a technique called "slotted page structure". This approach is used by many databases, like PostgreSQL.

We organise the page into a collection of slots or cells, with the pointers and cells being present on the opposite sides of the page.

Slotted page structure is as follows:

```
-------------------------------------------------------------------------------
------------------
```

```
| Header | Pointer to cell 1 | Pointer to cell 2 |----->>> Free Space  <<<-------|
Cell 2 | Cell 1 |
-------------------------------------------------------------------------------
------------------
```

We will look at an example of the above structure in a little while.

For now, let's understand some aspects of this page format that justifies the design decisions:

1. Problem: We need to store variable-size records with a minimal overhead. Solution: The only overhead in this approach is the "pointer to cell 1" part, that is, a pointer array.
2. Problem: We need to be able to reclaim space occupied by the removed records. Solution: Space can be reclaimed by defragmenting and rewriting the page.
3. Problem: We need to be able to reference records in the page without regard to their exact locations. Solution: Out-of-page pointers can reference the page and the slot IDs. They don't have to care about the exact location of the slot/cell. The exact location of the slot/cell is relative to the page.

Cell Layout

2 types of cells:

1. Key cells: They contain a separator key and a pointer to the page between two neighboring pointers.
2. Key-Value cells: They contain keys and data records associated with them.

Assumption: All cells in a page are of the same type. This allows us to store metadata of the cell only once in the page header, instead of storing it for each cell.

Key Cell

We need to know:

1. Cell type: This is inferred from the page header.
2. Key size
3. Page ID: This is ID of the page this cell is pointing to.
4. Key bytes

A variable size key cell can look like:

```
----------------------------------
| Key size | Page ID | Key bytes |
----------------------------------
```

Key-Value Cell

We need to know:

1. Cell type: This is inferred from the page header.
2. Key size

3. Value size
4. Key bytes
5. Value (data record) bytes

A variable size key-value cell can look like:

```
-----------------------------------------------------
| Key size | Value size | Key bytes | Value bytes |
-----------------------------------------------------
```

Combining Cells into Slotted Pages

We follow the structure of a slotted page that we learned above:

```
-------------------------------------------------------------------------------
------------------
| Header | Pointer to cell 1 | Pointer to cell 2 |----->>> Free Space  <<<-------|
Cell 2 | Cell 1 |
-------------------------------------------------------------------------------
------------------
```

Cell Offsets/Pointers: Pointer to cell 1, Pointer to cell 2, etc. are called cell offsets or cell pointers.

Keys can be inserted out of order, but the cell offsets are always kept in the order of keys. Why? Remember: Node of B-Tree == Page. The separator keys of the node are stored in sorted order to allow binary search. The cells essentially map to the separator keys.

Step by step example of a page that holds names

1. Insert "NameB"

```
-----------------------------------------
| Header | PointerB | Free Space | NameB |
-----------------------------------------
```

2. Insert "NameA"

```
-------------------------------------------------------------
| Header | PointerA | PointerB | Free Space | NameA | NameB |
-------------------------------------------------------------
```

Note that PointerA is before PointerB because NameA comes before NameB.

3. Insert "NameC"

```
-------------------------------------------------------------------------------
| Header | PointerA | PointerB | PointerC | Free Space | NameC | NameA | NameB |
-------------------------------------------------------------------------------
```

Managing Variable-Size Data

Removing a cell doesn't have to mean that we have to move all the subsequent cells. We can simply mark the cell as deleted and reuse the space when we need to insert a new cell.

When we mark a cell as deleted, an in-memory availability list is updated. This list contains the offsets of freed segments and their sizes. When inserting a new cell, we can use the space from the availability list if it's big enough.

If we cannot find a big enough space in the availability list, we can defragment the page by moving the cells to the beginning of the page and updating the cell offsets. This is an expensive operation, so we should do it only when we have to.

If there's still not enough free space after defragmentation, we have to create an Overflow page. We will learn about overflow pages in chapter 4.

Versioning

The binary data format might change over time, as the developers add new features or fix bugs. This can cause problems when we have to read old data using the new version of the software. Hence, most of the times, any storage engine has to support more than one serialization formats.

To figure out which serialization format to use, we make use of a version number, which helps determine the format of the data.

Example:

1. Apache Cassandra uses a version number prefix in the filenames.
2. PostgreSQL stores the version number in the PG_VERSION file.

After finding out the version of the file, we can create a version-specific reader to read the file contents.

Checksumming

1. Checksums provide weakest guarantee of data integrity, and cannot detect corruption in multiple bits.
2. CRCs can help detect corruption in multiple bits.

We update the checksum whenever we write a page to disk. When reading a page, we can verify the checksum to detect corruption.

Computing checksum over file contents is expensive, so we can compute it over the page contents and store it in the page header. This also means that we don't have to discard the whole file if the corruption is detected in one page.

Pending Doubts

1. On page 51 (in the General Principles section), we have an example showcasing how we can store a structure having a mix of fixed-size and variable-size fields. I have some confusion in that.

Let's say we have a structure like this:

```
struct Person {
    int age; // fixed size of say, 2 bytes
    char gender; // fixed size of say, 1 byte
    string name; // variable size where say, the number of characters is the
number of bytes required
}
```

And we have to store 2 persons:

- Person 1: (age, gender, name) = (10, M, Akshat)
- Person 2: (age, gender, name) = (15, M, Phil)

How would we store this? Is it like this?

```
Header: 2Age1Gender
Page 1: 10M6Akshat
Page 2: 15M4Phil
Trailer
```

The book seems to indicate that the variable-sized fields would also be part of the structure, but I am not sure how that would work. I think I am missing something here.

[Update] Answer by Alex Petrov (author): What I tried to say was that the structure holding the employee data can have first a block with fixed data followed by the variable size data.

2. On a similar note as question 1, what is the difference between "offset" and "length"? Context: The book says "To avoid calculations involving multiple fields, we can encode both offset and length to the fixed-size area. In this case, we can locate any variable-size field separately."

[Update] Offset = address in memory where the item starts Length = Length of the item in bytes.

3. Why is "[byte] flags" present in the representation of a Key-Value cell on page 55? What is the purpose of this? Why was it not present in the representation of a Key cell on page 54?

4. In Figure 3-9, it should be "Occupied cells are shown in gray." instead of "Occupied pages are shown in gray."?

[Update] Yes. It's a typo.

Things to Read

NA at the moment.

Reading group discussion

Nothing "extra" was discussed that is worth mentioning here.

Chapter 4: Implementing B-Trees

Page Header

Page header contains useful metadata like number of cells in the page, page contents and layout, etc.

- Postgres stores page size and layout version in the header.
- MySQL InnoDB stores number of heap records etc in the header.

Magic Numbers

Magic number is a multi-byte block placed in the header of the file or page. These are used to identify the file type and version.

✨ **Side learning:** For anyone interested in checking the different use-cases for which Postgres leverages magic numbers, I think this is a good place to start: PostgreSQL Code search for Magic Number

Sibling Links

For going from child1 to child2, we need to go to the parent and then to child2. This is a lot of work. So, we can add a link from child1 to child2. This is called a sibling link.

A sample tree showing sibling link:

```
        parent
      /       \
    child1 -> child2
```

Downside of sibling links: They need to be updated during splits and merges, so that's extra overhead.

Rightmost Pointers

A node having 1 separator key looks like:

```
[left child pointer] [separator key] [right child pointer]
```

Similarly, a node having multiple separator keys looks like:

```
p1 k1 p2 k2 p3 k3 p4
```

To be able to pair each separator key with its corresponding child pointer, we add a rightmost pointer to the node. This is a pointer to the rightmost child of the node.

So instead of the above representation, we get:

```
    [other stuff] p4 (p4 is stored separately in the page header)
    p1 k1 p2 k2 p3 k3
```

Node High Keys

This is an alternative to rightmost pointers. Instead of storing the rightmost pointer separately (to be able to pair the remaining keys with the pointers in a 1:1 fashion), we can add a high key to the node.

So instead of the above representation, we get:

```
p1 k1 p2 k2 p3 k3 p4 k4 (where k4 is the high key)
```

This simplifies handling edge cases.

Overflow pages

If a payload is too large to fit in a page, we can store it in an overflow page. This is a linked list of pages.

Most B-Tree implementations allow storing upto a fixed number of payload bytes in the page itself. If the payload is larger than that, it is stored in an overflow page.

Example:

```
Primary page:
    [payload1 with pointer to "payload 1 continued"]
    [payload2 with pointer to "payload 2 continued"]
    [free space in the page]

Overflow page:
    [payload1 continued]
    [payload2 continued]
    [free space in the page]
```

Overflow pages require some bookkeeping. Page ID of first overflow page is stored in the primary page. Each overflow page stores the page ID of the next overflow page (if the next page exists).

Binary Search

Insertion point: It is the index of the first element that is greater than the given key.

The binary search algorithm returns the insertion point if the key is not found in the array (since majority of searches are not going to result in exact matches).

Binary Search with Indirection Pointers

Since cell offsets are stored in a sorted way (as we learned in previous chapters), we pick the middle cell offset and compare it with the key we are searching for.

- If the key is greater than the middle cell offset, we pick the middle cell offset of the right half of the array and repeat the process.
- If the key is smaller than the middle cell offset, we pick the middle cell offset of the left half of the array and repeat the process.

Propagating Splits and Merges

B-Tree splits and merges can propagate up the tree. This is because the parent node needs to be updated with the new separator key and the new child pointer.

If we include parent pointers in the nodes, we can easily propagate splits and merges up the tree, but this adds extra overhead for changing parent pointers during splits and merges.

Breadcrumbs

Other option is to use a stack to keep track of the path from the root to the leaf node. This is a common approach used in B-Tree implementations, and is called breadcrumbing.

✨ **Side learning** (this one is unrelated to databases)

"Breadcrumb" is apparently a standard terminology not just specific to databases. For example: Breadcrumb navigation is a term used in web development. It is a navigation scheme that reveals the user's location in a website or web application. Source: Breadcrum Navigation

Breadcrumbs contain references to all the nodes traversed from the root to the leaf node. This is a stack of node pointers + insertion point in each node.

Rebalancing

Some implementations try to postpone splits and merges as much as possible, in order to amortize the cost.

This is done by:

1. rebalancing elements with the level
2. moving elements from higher occupancy nodes to lower occupancy nodes

This helps improving node occupancy, which reduces number of levels in a tree and helps prevent potentially higher cost of splits and merges.

B* trees are an example of such an implementation.

1. They keep distributing data between neighbouring nodes until all sibling nodes are full.
2. Instead of splitting a single node into 2 half nodes, they split 2 nodes into 3 nodes (each of them being 2/3 full).

Right-Only Appends

It's frequent to see database systems use auto-incremented IDs for primary keys. This means that all insertions will happen in the rightmost leaf, which also means that most of the splits would occur on the rightmost node

on each level. So, the idea here is to use that fact and think about optimizations.

Postgres calls this "fastpath".

SQLite has a similar concept and calls it "quickbalance".

Bulk Loading

If we have presorted data, and we want to bulk load it, or if we have to rebuilt the tree (for example, for defragmentation), we can leverage the idea of right-only appends and use it to optimise the process for bulk loading.

High-level steps:

1. We can compose the tree from bottom up, starting by writing presorted data on the leaf-node-level page-wise (we don't need to insert the elements one by one since we already know their order!).
2. After leaf-level is done, we can move up to the parent level and repeat the process.

Compression

Compression is a technique to reduce the size of the data. It is used to reduce the size of the data on disk, and also to reduce the amount of data that needs to be read from disk.

Tradeoff:

1. Larger compression ratios == less size on disk == less data to read from disk. But this also means that we need to spend more RAM and CPU cycles to compress and decompress the data.
2. Smaller compression ratios == more size on disk == more data to read from disk. But we need to spend less RAM and CPU cycles to compress and decompress the data.

Compression can be done at different levels:

1. files: This is impractical since we need to decompress the entire file to read a single page.
2. pages: This is a good approach, and allows coupling the compression/decopression with page loading/flushing. However, the compressed page now occupies only a fraction of a disk block, so any transfers would page in extra bytes (since transfers are usually done in units of disk blocks).
3. data (row-wise or column-wise): In this case, page loading/flushing is not coupled with compression/decompression.

Vacuum and Maintenance

- Addressable data records: Data records that can be reached by following pointers down from the root node. On a page level, cells that have cell pointers to them are addressable.
- Non-addressable data records: Data records that are not referenced anywhere and cannot be read or interpreted.

We have some processes that happen in the background to help us avoid the overhead during inserts/updates/deletes.

Fragmentation Caused by Updates and Deletes

Different cases:

1. Deletion of an element: On the leaf level, deletes only remove cell offsets from the page header. We don't need to zero-fill the cell itself, as eventually that area would be overwritten by new data anyway. And we don't need to worry about reading garbage data since we made it non-addressable by removing the cell offset from the page header.
2. Page split: During page splits, only the cell offsets are split, and we don't have to explicitly remove the cells from the page (that have been moved to the new page).

Page Defragmentation

Compaction / Vacuum: The maintenance process that takes care of space reclamation and page rewrites.

Page re-writes can be done synchronously or asynchronously.

Whenever pages are rewritten, they may also get relocated to new positions in the file.

IDs of the newly available on-disk pages are added to a "free page list" (also called freelist). This information has to be persisted to survive node crashes and restarts.

Pending Doubts

1. Are "magic numbers" similar to how we use prefixes to identify file types such as png, jpg, etc? Is that a good analogy for intuition?

2. In "Overflow Pages" section, it was written that: "Using this approach, we cannot end up in a situation where the page has no free space, as it will always have at least max_payload_size bytes." --- Why? How?

3. In "Bulk Loading" section, it was mentioned that: "Immutable B-Trees can be created in the same manner but, unlike mutable B-Trees, they require no space overhead for subsequent modifications, since all operations on a tree are final. All pages can be completely filled up, improving occupancy and resulting into better performance." --- What does the following things mean?

    - "they require no space overhead for subsequent modifications" - what space overhead are we talking about here?
    - "All pages can be completely filled up, improving occupancy and resulting into better performance" - what does mutability have to do with occupancy?

4. In "Page Defragmentation" section, it was mentioned that: "Whenever pages are rewritten, they may also get relocated to new positions in the file." --- Why can we not always do them in-place?

5. In "Page Defragmentation" section, it was mentioned that: "Page rewrites can be done synchronously on write if the page does not have enough free physical space (to avoid creating unnecessary overflow pages)" --- Why do we need to avoid creating unnecessary overflow pages?

Things to Read

1. Use-cases of immutable B-Trees

Reading group discussion

This section contains anything worth mentioning that came up as part of the weekly reading group discussion. I will update it when I am adding notes for the next chapter.

Chapter 5: Transaction Processing and Recovery

Transaction: Indivisible unit of work that either succeeds completely or fails completely.

✨ **Side learning:** Transactions are not locks

ACID properties

- Atomicity: All or nothing. Either all the operations in a transaction succeed, or none of them do.
- Consistency: The database is always in a valid state. This is the most weakly defined property, because it's the only property that is controller by the user and not by the database itself.
- Isolation: Multiple transactions that are running concurrently should not interfere with each other (as if no other transaction is running at the same time).
- Durability: Once a transaction has completed successfully, the changes it has made to the database are permanent. This allows the database to recover from failures.

A bunch of different components come together to implement transactions. These are:

1. Transaction manager: Coordinates, schedules, and tracks transactions.
2. Lock manager: Prevents concurrent access to data. If the lock is exclusive, then only one transaction can access the data at a time, and other transactions have to wait for the lock to be released. If the lock is shared, then multiple transactions can read the data at the same time.
3. Page cache: Intermediary between the database's storage engine and the persistent storage (disk). It manages caching of pages in memory.
4. Log manager: Writes all the changes to the database to a log file. This is used for recovery in case of a crash.

Buffer Management

Goal is to reduce the number of accesses to the disk, because disk access is slow. For this, pages are cached in memory.

Virtual Disk: A read from virtual disk accesses physical storage (disk) only if the page is not in the page cache. If the page is in the buffer pool, then it is read from there.

Virtual Disk / Page Cache / Buffer Pool: All these terms are used interchangeably. We will use the term page cache for the rest of the notes.

Some other phrases that are widely used:

1. "Paged in": When a page is read from the disk (because it is not in cache), it is said to be paged in.
2. Dirty page: A page that has been modified in memory but not yet written to disk.
3. "flushed to disk": When a page is written to disk, it is said to be flushed to disk.
4. "evicted": When a page is removed from the page cache, it is said to be evicted.

Caching Semantics

When storage engine requests the page:

1. If the page is in the page cache, then it is returned.
2. If the page is not in the page cache, then it is read from the disk and returned.

Until the storage engine hands over the page to the page cache, the page is said to be "refenced". Once the page is handed over to the page cache, the page is said to be "unreferenced".

The page cache can be told to avoid evicting a page by "pinning" it.

Cache Eviction

Page cache has a limited capacity. When the page cache is full, it needs to evict some pages to make space for new pages.

1. Dirty pages need to be flushed to disk before they can be evicted.
2. Referenced pages cannot be evicted while some thread is still using them.

Postgres has a background flush writer that cycles over the dirty pages that are likely to be evicted, and flushes them to disk. This is because triggering a flush on every eviction might be bad for performance.

Durability: If the database crashes, all the dirty pages that have not been flushed to disk are lost. To ensure durability, the checkpoint process coordinates the page cache and the write-ahead log (WAL).

Which log records can be discarded from WAL? Only the ones which correspond to operations applied to cached pages that have been flushed to disk. This is because the changes are already on disk, so they can be recovered from there in the case of a crash.

Locking Pages in Cache

Pinning: Locking pages in the cache so that they cannot be evicted.

Which pages would we want to pin? The ones that we know we will need to access again soon.

In the context of B-Trees, we would want to pin the pages that are on the higher-levels of the B-Tree, because they are more likely to be accessed again soon. The higher-levels have exponentially lesser nodes than the lower-levels, so it is feasible to cache them. In the case of splits or merges, the changes might propagate to the higher-levels, so having the higher-levels in cache is useful and help reduce the number of disk accesses.

Another good aspect here is that multiple writes to the same page can be batched together, and the page can be flushed to disk only once. For eaxmple, if 10 splits or merges propagate to the root node, then all of them can be batched together and the page can be flushed to disk only once.

Prefetching and Immediate Eviction

Prefetching: In some cases, when a page is read from disk, we can already tell which pages would be likely to be accessed soon. Example, we can load the next leaf nodes during a range scan.

Immediate Eviction: In some cases, we can evict a page from the cache as soon as it has been read from disk. Example, if a maintenance process loads the page, it's unlikely that the page would be useful for any future queries, so it can be evicted immediately.

Page Replacement

Page replacement: When the page cache is full, and a new page needs to be loaded, then some existing page needs to be evicted to make space for the new page.

Page replacement policy / Eviction policy: The algorithm that decides which page to evict.

In an ideal world, we would want to evict the pages that would not be touched for the longest time in the future. But, this is not possible to predict.

Belady's Anomaly: Using larger cache might not always reduce the number of evictions. Belady's anomaly shows that increasing the number of pages might still increase the number of evictions in some cases, if the eviction policy is not good.

We discuss some eviction policies below.

FIFO and LRU

FIFO == First In First Out LRU == Least Recently Used

FIFO eviction policy: Evict the page that was loaded first. LRU eviction policy: Evict the page that was accessed least recently.

LRU-K eviction policy: Evict the page that was accessed least recently, but taking only the last K accesses into account.

Clock

Clock sweep algorithm holds references to pages and an "access bit" in a circular buffer.

The clock hand moves around the clock, and the page that it points to is a candidate for eviction.

1. If access bit is 1, and the page is unreferenced, then the access bit is set to 0 and the clock hand moves to the next page.
2. If access bit is 0, then the page becomes a candidate and is scheduled for eviction.
3. If the page is currently referenced, don't change the access bit.

Access Bit	Is Referenced	Action
1	Unreferenced	Set access bit to 0 and move the clock hand to the next page
0	-	Page becomes a candidate for eviction and is scheduled for eviction
-	Referenced	Do not change the access bit

LFU

LFU == Least Frequently Used

TinyLFU is a frequency based page eviction policy that uses this.

Recovery

Write-Ahead-Log (WAL) (also known as commit log) == Append-only auxiliary disk-resident structure used for crash and recovery.

WAL functionalities:

1. Allows page cache to buffer updates to disk-resident pages.
2. Persist all operations on disk until the caches copied of pages are flushed to disk.
3. Allow lost in-memory changes to be reconstructed from the log in the case of failures.

With the above functionalities, it plays an important role in transaction processing. In the case of crash, uncommited data is replayed from the log and the pre-crash state is restored.

Log Semantics

WAL consists of a sequence of log records. Each log record contains a unique, monotonically increasing LSN (log-sequence-number) that identifies the log record.

Contents of log records are cached in the log buffer, and are flushed to disk in a `force` operation. Flushing happens in the sequence of LSNs.

WAL also holds records that indicate the start and end of a transaction. A transaction cannot be considered committed until the log is forced to the LSN of the commit record.

Compensation Log Records (CLRs): To be able to recover from crashes **during rollback or recovery**, we use CLR. Good link explaining CLRs: https://dba.stackexchange.com/a/299446

Checkpoints

Checkpoints are used to reduce the amount of work that needs to be done during recovery. Sync checkpoint is a process to force all dirty pages to be flushed to disk.

But, flushing the entire contents on disk is impractical. So, we use fuzzy checkpoints. A fuzzy checkpoint begins with a `begin_checkpoint` record, and ends with an `end_checkpoint` record, and contains information about the dirty pages and transaction. Once pages are flushed (asynchronously), the `last_checkpoint` pointer is updated to point to the `begin_checkpoint` record.

Operation Versus Data Log

Shadow paging: It's a copy-on-write technique to ensure transaction atomicity by placing new contents in a new unpublished "shadow page", which are made visible by toggling a pointer (to point it to the new page).

We can use 2 types of logs:

1. Physical log: Stores complete page state.
2. Logical log: Stores only the operations.

Practically, databases use a combination of both:

1. Physical logging to perform a redo (since that improves recovery time)
2. Logical logging to perform an undo (since that improves concurrency and performance)

Steal and Force Policies

Databases define steal/no-steal and force/no-force policies to determine "when" the in-memory changes are flushed to disk.

Steal Policy:

- Steal policy: Allows dirty pages to be flushed to disk before the transaction commits.
- No-steal policy: Dirty pages are not allowed to be flushed to disk before the transaction commits.
- "Stealing" a dirty page means flushing it to disk and replacing it with a different page.

Force Policy:

- Force policy: Requires dirty pages to be flushed to disk "IMMEDIATELY BEFORE" the transaction commits.
- No-force policy: Allows transaction to commit even if the dirty pages have not been flushed to disk.
- "Forcing" a dirty page means flushing it to disk before the commit.

I found the explanation in the book quite confusing. I found a much better explanation here that cleared things for me: https://stackoverflow.com/a/37861999

Why are we discussing these policies in the context of recovery?

- Using no-steal policy allows implementing recovery using only redo entries.
- Using no-force policy allows us to buffer several updates to pages in the page cache, and flush them to disk in a single batch. This improves performance.

General rule of thumb:

- If any pages touched by a transaction are flushed, we need to keep "undo" information in the log until it commits, so that we can use that to roll the transaction back.
- If any pages touched by a transaction are not flushed, we need to keep "redo" information in the log until it commits, so that we can use that to reach the pre-crash state.

ARIES

ARIES == Algorithm for Recovery and Isolation Exploiting Semantics

It is a steal/no-force recovery algorithm.

It uses:

1. physical redo to improve performance during recovery.
2. logical undo to improve performance during normal operation (since they can be applied to pages independently).
3. WAL records to implement repeating history during recovery.

When database restarts after the crash, recovery happens in 3 phases:

1. Analysis Phase: Identify the dirty pages and the transactions that were active at the time of the crash.
2. Redo Phase: Repeats the history and restores the databse to the state it was in at the time of the crash. This phase is done for both committed and uncommitted transactions, whose contents were not flushed to disk.
3. Undo Phase: Rolls back the uncommitted transactions in reverse chronological order.

Concurrency Control

3 types of techniques to handle concurrent transactions:

1. Optimistic concurrency control (OCC): Transactions do not block each other. If execution results in conflict, one of the transactions is aborted.
2. Pessimistic concurrency control (PCC): Prevents conflicts by locking data.
3. Multi-version concurrency control (MVCC): Allows multiple versions of the same data to exist at the same time.

✨ **Side learning:**

I found the above explanation from the book quite confusing. I found a much better explanation here that cleared things for me: https://stackoverflow.com/a/39269085

TLDR of the above link:

1. Optimistic Concurrency Control: This approach assumes that conflicting operations do not happen frequently (that is, it's optimistic!). This is why it allows transactions to proceed without locking the data. If a conflict happens, then one of the transactions is aborted.
2. Pessimistic Concurrency Control: This approach assumes that conflicting operations happen frequently (that's why it's called pessimistic). This is why it locks the data before allowing transactions to proceed.
3. Multi-version concurrency control (MVCC): This is a concurrency control method that typically belong in the category of Optimistic Concurrency Control. It is theoretically possible to combine it with pessimistic methods (like locking), but its multi-version nature is best combined with optimistic methods.

Serializability

Schedule == List of operations required to execute a set of transactions from database's perspective.

"Complete" schedule: A schedule that contains all the operations of all the transactions.

"Correct" schedule: If the schedule is logically equivalent to the original lists of operations, but their parts can be executed in parallel or reordered.

"Serial" schedule: A schedule in which the operations of each transaction are executed consecutively without interleaving. This significantly reduces the concurrency, hence reducing throughput.

Serializability: A schedule is serializable if it is equivalent to "some" serial schedule.

For example, if we have 3 transactions T1, T2, and T3, then the following are the possible serial schedules:

1. T1, T2, T3
2. T1, T3, T2
3. T2, T1, T3
4. T2, T3, T1
5. T3, T1, T2
6. T3, T2, T1

Transaction Isolation

"Isolation level" defines the degree to which the operations of a transaction are visible to other transactions.

Read and Write Anomalies

3 types of read anomalies:

1. Dirty read: A transaction reads a value that has not yet been committed.
2. Non-repeatable read: A transaction reads a value, and then reads the same value again, but gets a different value the second time.
3. Phantom read: A transaction reads a set of rows that satisfy a search condition, and then repeats the search, but the set of rows that satisfy the search condition has changed in the meantime. It is similar to non-repeatable read, but it's for range queries.

3 types of write anomalies:

1. Lost update: A transaction overwrites the changes made by another transaction. Hence, the updates made by the other transaction are lost.
2. Dirty write: A transaction reads an uncommitted value (that is, dirty read), modifies it, and then commits it. Hence, transaction results are based on uncommitted data.
3. Write skew: When each individual transaction respects the invariant, but the invariant is violated when the transactions are considered together.

Isolation Levels

Isolation Level	Dirty	Non-Repeatable	Phantom
Read Uncommitted	☑	☑	☑
Read Committed	✕	☑	☑
Repeatable Read	✕	✕	☑
Serializable	✕	✕	✕

☑ means that the anomaly is allowed for that isolation level.

✕ means that the anomaly is not allowed for that isolation level.

Snapshot Isolation

Snapshot isolation allows transactions to read data as it existed at the start of the transaction. Each transaction takes a snapshot of the database at the start of the transaction, and reads from that snapshot for the rest of the transaction. The transaction commits only if the values it has modified have not been modified by any other transaction. If the values have been modified, then the transaction is aborted and rolled back.

This makes "lost update" anomaly impossible, because the transaction is aborted if the values it has modified have been modified by another transaction.

However, a "write skew" anomaly is still possible.

Optimistic Concurrency Control

Optimistic concurrency control (OCC) assumes that conflicting operations do not happen frequently.

3 phases of transaction execution:

1. Read Phase: Transaction reads the data it needs. After this step, all transaction dependencies (read set) and the output (write set) are known.
2. Validation Phase: Determines whether committing the transaction preserves ACID properties.
3. Write Phase: If validation phase has been successful, transaction commits the "write set" to the database.

"Validation" is done by checking for conflicts with other transactions.

1. Backward oriented: Checks conflicts with transactions that have already been committed.
2. Forward oriented: Checks conflicts with transactions that are in the validation phase.

Multiversion Concurrency Control

MVCC distinguishes between committed and uncommitted versions of the data. It allows multiple versions of the same data to exist at the same time.

Major use case for MVCC is snapshot isolation.

Pessimistic Concurrency Control

This assumes that conflicting operations happen frequently, hence is more conservative than optimistic concurrency control.

Timestamp ordering

This is one of the simplest pessimistic lock-free concurrency control schemes.

The transaction manager maintains `max_read_timestamp` and `max_write_timestamp` for each value.

When a transaction reads a value, `max_read_timestamp` is updated. When a transaction writes a value, it sets `max_write_timestamp` is updated.

- Read operations that try to read a value with a timestamp that is less than `max_write_timestamp` are aborted, since the value has been modified by a transaction that started after the current transaction.
- Write operations that try to write a value with a timestamp that is less than `max_read_timestamp` are aborted, since the value has been read by a transaction that started after the current transaction.

Lock-Based Concurrency Control

Lock-based concurrency control is a pessimistic concurrency control scheme that uses locks on database objects (instead of trying to resolve schedules).

Two-phase locking: This separates the locking phase from the unlocking phase.

1. Locking / Growing / Expanding phase: Acquire all the locks that are needed by the transaction. **No locks are released.**
2. Unlocking / Shrinking phase: Release all the locks that were acquired by the transaction during the growing phase. **No locks are acquired.**

Deadlocks: A situation when two or more transactions are waiting for each other to release the locks. Example, T1 has locked A and is waiting for B, and T2 has locked B and is waiting for A. So both are forever waiting for each other to release the locks, hence are said to be "in deadlock".

How to handle deadlocks? --- Introduce timeouts and abort long-running transactions.

How to detect deadlocks? --- Using wait-for graphs.

1. This is a directed graph where each transaction is a node, and there is an edge from T1 to T2 if T1 is waiting for T2 to release the lock. If there is a cycle in the graph, then there is a deadlock.
2. Deadlock detection can be done periodically, or everytime the wait-for graph is updated.

How to avoid deadlocks? --- By restricting lock acquisition. Transaction manager can use timestamps to determine priority order of transactions. A lower timestamp means higher priority.

If T1 attempts to acquire a lock that is held by T2, and T1 started before T2 (that is, T1 has higher priority), then we can use one of the following:

1. Wait-die: T1 waits for T2 to release the lock.
2. Wound-wait: T1 aborts T2 and acquires the lock.

Locks

- Locks are used to isolate and schedule transactions and manage database contents, NOT the internal storage structure itself.
- Locks are acquired on keys.
- Locks are generally stored and managed outside of the tree implementation.
- Locks are more heavyweight than latches.
- Locks are held for the duration of the transaction.

Latches

- Latches guard the internal storage structure, that is, the physical representation.
- Latches are acquired on pages.
- Even the lockless concurrency control techniques have to use latches.
- Any page has to be latched for safe concurrent access.

Example use-case: Modifications on leaf-level can propagate to higher levels of B-Tree. In such cases, latches are obtained on multiple levels. This is to ensure that executing queries on the higher levels does not result in inconsistent data (such as incomplete writes or partial node splits).

Readers-Writer Lock

Readers-Writer Lock (RW Lock) is a lock that allows concurrent (shared) read access, but exclusive write access.

Latch Crabbing

Latch crabbing (also called latch coupling) is an optimisation technique to minimise the time for which a latch is held. This is especially useful when we have a simple implementation for latches, where we obtain latches on all the pages from the root to the target leaf node.

Latch crabbing allows:

1. Holding latches for less time
2. Releasing them when the executing operation doesn't need them anymore

When can the latch be released?

1. On a read path, the latch on parent node can be released once the child node has been located.
2. On an insert path, the latch on parent node can be released if the operation is not going to modify the parent node (via splits).
3. On a delete path, the latch on parent node can be released if the operation is not going to modify the parent node (via merges).

Latch Upgrading and Pointer Chasing

Latch upgrading: Instead of acquiring exclusive locks during traversal, we can acquire shared locks and then upgrade them to exclusive locks when needed.

For example: Write operations on leaf nodes require exclusive locks. But, we can acquire shared locks on the parent nodes, and then upgrade them to exclusive locks "if needed" (due to splits or merges).

If multiple threads are trying to upgrade the same latch, then only one of them can succeed. The other threads have to wait or restart.

Pointer chasing: Every request goes through the root node, so we can always latch the root node optimistically. Price of "retry" (pointer chasing) is very rarely paid for root nodes since splits/merges rarely propagate to the root node.

Blink-Trees

Blink-Trees allow a state called "half-split" where a node is referenced by sibling pointer, but not by the parent's child pointer.

Advantage: We don't need to hold parent lock when descending to child level, even if it's going to lead to a split. The new node can be made visible through the sibling link, and we can let the parent pointer update lazily without sacrificing correctness. This prevents the search path from being blocked by the parent node, causing unnecessary restarts, as all elements in the tree are accessible.

Pending Doubts

1. In the "Log Semantics" section (Page 91), it's mentioned that "Pages are flushed asynchronously and, once this is done, the last_checkpoint record is updated with the LSN of the begin_checkpoint record and, in case of a crash, the recovery process will start from there"

Shouldn't the "last_checkpoint" record be updated with the LSN of the "end_checkpoint" record?

2. In "Latch Crabbing" section, it was mentioned that "If the child page is still not loaded in the page cache, we can either latch a future loading page, or release a parent latch and restart the root-to-leaf pass after the page is loaded to reduce contention."

What does it mean?

3. In "Latch Upgrading and Pointer Chasing" section, it was mentioned that "The root is always the last to be split, since all of its children have to fill up first. This means that the root node can always be latched optimistically, and the price of a retry (pointer chasing) is seldom paid."

Does "the root node can always be latched optimistically" mean that all threads can be allowed to acquire a "shared latch" on the root node? If not, what does it mean?

Things to Read

1. Franck Pachot posted a blog: Isolation Levels in Modern SQL Databases
2. Can't wait to read WAL and Lock related content in PostgreSQL 14 Internals book.

Reading group discussion

This section contains anything worth mentioning that came up as part of the weekly reading group discussion. I will update it when I am adding notes for the next chapter.

Chapter 6: B-Tree Variants

In this chapter, we discuss some of the variants of B-Trees.

Copy-on-Write

Copy-on-Write B-Trees use copy-on-write technique to ensure data integrity during concurrent operations.

- Whenever a page has to be updated, its contents are copied, the copy is updated. This way a paralle tree hierarchy is created.
- Old/Original/Initial versions of tree still remain accessible to readers (that happen concurrently with the writers).
- Once the writer is done with the update, it updates the root pointer to point to the new tree.

Downsides:

- Requires more space.
- Requires more processor time.

Advantages:

- Readers require no synchronization.
- Writers are not blocked by readers.
- Crash cannot leave the tree in a corrupted state, because we point to the "copy" only after the update is complete.

Implementing Copy-on-Write: LMDB

- LMDB == Lightning Memory-Mapped Database == a storage engine that uses copy-on-write.
- LMDB is a key-value store used by OpenLDAP.
- LMDB holds only 2 versions of the tree at any given time.

Abstracting Node Updates

Few ways to represent a node in-memory:

1. Direct Access to Cached Version or Native Memory Manipulation
2. Materializing Nodes into Language-Native Structures
3. Wrapper Object Approach in Managed Memory Languages

Lazy B-Trees

Lazy B-Trees use update-friendly in-memory structures to buffer updates and propagate them with a delay.

WiredTiger

WiredTiger is the default storage engine for MongoDB.

B-Tree nodes are materialized in memory upon paging in, storing updates until a flush is triggered.

Page Structure:

- In WiredTiger, a clean page initially consists of an index constructed from the on-disk page image.
- Updates are saved into the update buffer, accessed during reads to merge with the original on-disk page contents.

Reconciliation Process:

- Before in-memory pages are persisted, they undergo a reconciliation process.
- During flush, update buffer contents are reconciled with page contents and overwritten on disk; large pages are split if needed.

Implementation Details::

- Update buffers in WiredTiger are implemented using skiplists, offering a concurrency profile advantage.
- Clean and dirty pages both have in-memory versions referencing a base image on disk, with dirty pages additionally having an update buffer.

Lazy-Adaptive Tree

- LA-Tree groups nodes into subtrees and attaches an update buffer to each subtree for batching operations.
- Update buffers track operations on the subtree top node and its descendants.

Insertion:

- When inserting a data record, a new entry is added to the root node update buffer.
- When the buffer is full, changes are copied and propagated to buffers in lower tree levels recursively until reaching leaf nodes.

Batched Updates for Efficiency:

- LA-Tree optimizes update time by performing batched updates, minimizing disk accesses and structural changes.
- Splits and merges propagate in batches to higher levels, reducing the need for separate updates on individual pages.

FD-Trees

FD-Trees == Flash-Drive Trees

Idea here is to group the "updates that are targeted different nodes" together, by using append-only storage and merge processes.

FD-Tree Design:

- Small mutable head tree and multiple immutable sorted runs.
- Minimizes the surface area where random write I/O is required.

Insertion:

- Head tree acts as a buffer in-place for any updates.
- When the head tree is full, the contents are transferred to the immutable run.
- Runs are merged with the next level if they exceed a threshold.

Fractional Cascading

Goal: To reduce the cost of locating an item in the cascade of sorted arrays.

Approach: Build bridges between the different levels, to facilitate starting the search from the closest match on the previous level.

For example, if we have multiple sorted arrays:

```
A1 = [12, 24, 32, 34, 39]
A2 = [22, 25, 28, 30, 35]
A3 = [11, 16, 24, 26, 30]
```

We can bridge the gaps by pulling every alternate element to an upper level:

```
A1 = [12, 24, 25, 30, 32, 34, 39]
A2 = [16, 22, 25, 26, 28, 30, 35]
A3 = [11, 16, 24, 26, 30]
```

Logarithmic Runs

An FD-Tree == fractional cascading + logarithmically sized sorted "runs"

`logarithmically sized sorted "runs"` == immutable sorted arrays with sizes increasing by a factor of k, which are created by merging the previous level with the current one.

Addressing Items in Sorted Runs:

FD-Trees adapt fractional cascading for addressing items in sorted runs.

Reduced Search Cost:

Head elements from lower levels act as pointers to higher levels. Reduces search cost in lower-level trees by leveraging previous searches from higher levels.

Bw-Trees

3 problems that we need to solve:

1. Write amplification
2. Space amplification
3. Complexity in solving concurrency problems + dealing with latches

Solution: Bw-Trees!

Bw-Tree (Buzzword-Tree):

- Uses append-only storage to batch updates to various nodes that are linked together in chains, optimizing write operations.
- Achieves a lock-free structure by linking nodes into chains and employing an in-memory data structure that enables pointer installation between nodes with a single compare-and-swap operation.

Update Chains

Bw-Tree adopts an update chain strategy, separating base and delta nodes in a linked list, allowing individual updates without rewriting nodes, but requires traversing all delta nodes during reads to reconstruct the node state, akin to LA-Trees.

Taming Concurrency with Compare-and-Swap

Algorithm to update a Bw-Tree node:

1. Locate the target logical leaf node by traversing the tree from root to leaf, utilizing virtual links in the mapping table to access target base nodes or the latest delta nodes in the update chain.
2. Create a new delta node with a pointer to the identified base node (or the latest delta node) from step 1.
3. Update the mapping table with a pointer to the newly created delta node, ensuring the algorithm tracks the most recent modifications in the update chain.

Structural Modification Operations

Bw-Tree also requires splits and merges. We call them Structural Modification Operations (SMOs).

Split SMOs in Bw-Tree: Start by consolidating logical contents of the splitting node, creating a new page to the right of the split point. Two-step process:

1. Split: Append a special split delta node to the splitting node with a midpoint separator key and a link to the new logical sibling node.
2. Parent Update: Add the new sibling node as a direct child to the parent node, bypassing the need to traverse through the splitting node, completing the split process.

Merge SMOs in Bw-Tree:

1. Remove Sibling: Create a special remove delta node appended to the right sibling, marking it for deletion and initiating the merge process.
2. Merge: Generate a merge delta node on the left sibling, pointing to the contents of the right sibling and integrating it logically.
3. Parent Update: Remove the link to the right sibling from the parent node to complete the merge, making the right sibling's contents accessible from the left sibling.

Consolidation and Garbage Collection

Epoch-Based Reclamation technique:

1. Bw-Trees use an Epoch-Based Reclamation technique to separate threads that might have encountered a specific node from those that couldn't.
2. Nodes and deltas removed from the mapping table during consolidations are preserved until every reader that started during the same or earlier epoch finishes, ensuring safe garbage collection afterward as later readers are guaranteed not to have seen those nodes.

Cache-Oblivious B-Trees

Cache-Oblivious B-Trees:

- Asymptotically optimal B-Tree performance without tuning parameters, adapting to diverse memory hierarchies.

Two-Level Memory Hierarchy:

- Traditional B-Trees for disk-resident pages and a limited-size page cache, with data transfer in blocks.

Simplicity in Parameterization:

- Design simplicity with only two parameters (page cache and disk) and cache-aware block transfers.

Cache-Oblivious Algorithms:

- Optimal performance without platform-specific parameters, maximizing work at the highest cache level.

van Emde Boas Layout

van Emde Boas Layout and Dynamic Operations:

- Cache-oblivious B-Trees use van Emde Boas layout for a static B-Tree and a packed array structure for dynamic operations.
- The van Emde Boas layout ensures contiguous memory storage, and the packed array, with density-based gaps, allows efficient inserts and updates.

Challenges and Future Considerations:

- On-disk structures mimic main memory layouts, but challenges include potential impacts from abstracted cache loading and comparable block transfer complexity to cache-aware counterparts.
- Future improvements may arise with the prevalence of more efficient nonvolatile byte-addressable storage devices.

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

1. Andy Pavlo has a really good paper on implementing Bw-trees: Link

Chapter 7: Log-Structured Storage

In this chapter, we will focus on LSM Trees.

LSM Trees

LSM Tree uses buffering + append-only storage to achieve sequential writes.

Why is it called LSM Tree?

- The name comes from log-structured filesystems, which write all modifications on disk in a log-like file.
- "Merge" in LSM Trees == Tree contents are merged using an approach like merge-sort.

Write performance and throughput

- Files are immutable.
- So, insert/update/delete operations don't need to locate data on disk.
- This improves write performance and throughput.

Reads

- Reads and writes don't intersect, thanks to the design.
- This simplifies concurrent access.

LSM Tree Structure

- Memtable: Memory-resident mutable component to buffer data records, and serves as a target for read and write operations.
- Write-Ahead Log (WAL): This is required to guarantee durability of data records. Data records are appended to the log before committing to the memtable.
- Disk-resident components: Used only for reads. We will call these "tables" in this chapter (as a shortcut for disk-resident tables).

Two-component LSM Tree

This has two components:

- Memtable
- Only one disk-resident component, comprised of immutable segments.

Memory-resident tree contents are flushed on disk in parts.

Multicomponent LSM Trees

Alternative design: Having more than one disk-resident table.

Compaction: To keep the number of disk-resident tables under control, LSM Trees periodically merge the disk-resident tables. This process is called compaction.

Memtable:

- In this case, entire memtable is flushed to disk, because we are allowed to have more than one disk-resident table. This simplifies compaction process.
- Memtable flushes can be triggered by size, or periodically.

Memtable Flush - The Process:

- **Memtable Switch:**
    - A new memtable is allocated.
    - The switch is performed atomically to ensure consistency.
    - The new memtable becomes the target for all new writes.
- **Flushing State:**
    - The old memtable, now in a flushing state, retains its contents.
    - It remains available for reads until its contents are fully flushed to disk.
- **Disk-Resident Table:**
    - The flushed data is written to disk as a new table (e.g., SSTable).
    - The old memtable is discarded.
- **Availability for Reads:**
    - Once fully flushed, the disk-resident table becomes available for reads.
- **After Flushing:**
    - The write-ahead log can be trimmed.
    - The log section holding operations-applied-to-the-flushed-memtable can be discarded.

The responsibilities of the different components involved in this process are very nicely summarised in the book. I am copy pasting it here for reference:

- **Current Memtable:** Receives writes and serves reads.
- **Flushing Memtable:** Available for reads.
- **On-Disk Flush Target:** Does not participate in reads, as its contents are incomplete.
- **Flushed Tables:** Available for reads as soon as the flushed memtable is discarded.
- **Compacting Tables:** Currently merging disk-resident tables.

- **Compacted Tables:** Created from flushed or other compacted tables.

Updates and Deletes

Updates: Just append, and let it reconcile the redundant records during reads.

Deletes: We need to use tombstones to mark the deleted records.

What is a tombstone?

- It is a special "delete entry".
- Tombstones help prevent resurrection of deleted records during reads.
- We can also have range tombstones, which are used to mark deletion of a range of records.

LSM Tree Lookups

Merge-Iteration

- Contents of disk-resident tables are sorted.
- This allows us to use multiway merge-sort algorithm (which uses a priority queue such as a min-heap).

Since the disk-resident tables are sorted, we can be sure that only the latest version for a key would be eventually included in the merged result.

Reconciliation

To be able to reconcile the redundant records, we need to have a way to identify the latest version of a record (the version that has to take precedence).

Data records hold metadata necessary for this, such as timestamps.

Maintenance in LSM Trees

- Periodic compaction is needed to reduce the growing number of disk-resident tables.
- Compaction picks multiple disk-resident tables, iterates over their entire contents using the (merge + reconciliation) algorithms, and writes out the results into the newly created table.

Leveled Compaction:

- Diks-resident tables separated into levels with target sizes.
- Level-0 tables from flushed memtable contents.
- Higher-level tables created through compaction of overlapping key ranges.
- Exponential size growth between levels, freshest data in lower levels.

Size-Tiered Compaction:

- Tables grouped by size, not levels.
- Level 0 holds smallest tables, incremented recursively with compaction.
- Larger tables promoted, smaller tables demoted.

Other Compaction Strategies:

- Apache Cassandra's time window compaction strategy for time-series workloads.

- Considers write timestamps and allows dropping files with expired data without rewriting contents.

Read, Write, and Space Amplification

We have 3 problems when storing data on disk in an immutable manner:

- **Read amplification:** Resulting from a need to address multiple tables to retrieve data.
- **Write amplification:** Caused by continuous rewrites by the compaction process.
- **Space amplification:** Arising from storing multiple records associated with the same key.

RUM Conjecture

RUM == Read, Update, and Memory overheads

RUM Conjecture states that reducing 2/3 of the overheads leads to the third one increasing.

- B-Trees excel in read efficiency but incur higher update and memory overheads.
- LSM Trees optimize updates and tend to have lower memory overhead but may have higher read overheads.

Implementation Details

This section covers how the disk-resident tables are implemented.

Sorted String Tables

Sorted String Tables == SSTables == data records are sorted in the key order.

SSTables generally have 2 components:

1. Index files: Holds keys and offsets of the corresponding data records in the data files. Implemented using B-Trees or hash tables.
2. Data files: Holds the data records in key order.

Bloom Filters

The root cause of "Read Amplification" problem in LSM Trees is that we need to check multiple disk-resident tables for a complete read operation. This can be improved by using Bloom Filters.

- Bloom Filters are probabilistic data structures that can tell us whether a key is present in a set or not.
- They either tell us that a key is either "absent" or "possibly present".
- A bloom filter uses a bit array and a set of hash functions.
- A "set bit" / "1 bit" means that the key is "possibly present". A "clear bit" / "0 bit" means that the key is "absent".

For example, if we have a bloom filter with 10 bits, and 2 hash functions, and we have the following:

HashFunction1("key1") = 3 HashFunction2("key1") = 7

Inserting "key1" into the bloom filter would result in the following:

```
[0, 0, 0, 1, 0, 0, 0, 1, 0, 0]
```

Similarly, for "key2", let's say we have:

HashFunction1("key2") = 1 HashFunction2("key2") = 4

Inserting "key2" into the bloom filter would result in the following:

```
[0, 1, 0, 1, 1, 0, 0, 1, 0, 0]
```

Now, for "key3", let's say we have: HashFunction1("key3") = 1 HashFunction2("key3") = 5

To check if "key3" is present in the bloom filter, we would do the following:

- Check if the bit at index 1 is set. It is set, so we move on to the next step. This is because we have a "possibly present" result, since it's not guaranteed that the bit was set because of "key3".
- Check if the bit at index 5 is set. It is not set, so we can conclude that "key3" is not present in the bloom filter.

In the context of LSM Trees, each disk-resident table can have a bloom filter attached to it, which can be used to check if a key is "possibly present" in the table or "not present". This can help us avoid reading the table if the key is not present in it.

Skiplist

Skiplist is like a linked list, but with additional pointers that allow us to skip some elements. A skiplist consists of a series of nodes of a different height, building linked hierarchies allowing to skip ranges of items.

For example:

```
|  | --------------------------------> |    |
|  | ----------> | 5 | --------------> | 10 |
|  | -> | 3 | -> |   | -> | 7 | -----> |    |
```

If we have to search for key 7:

1. We follow the pointer on the highest level to node 10.
2. Since search key (7) is smaller than 10, we follow the next-level pointer from the previous node, that is, to node 5.
3. We follow the highest level pointer from node 5 to node 10.
4. Since search key (7) is smaller than 10, we follow the next-level pointer from the previous node, that is, to node 7.

Disk Access

- LSM Trees leverage the page cache for efficient disk access, and in-memory data is made immutable, eliminating the need for locks.
- Reference counting ensures that actively used pages are not removed from memory during compaction.
- Unlike traditional structures, LSM Trees allow data records to span across page boundaries.

Compression

- Compression in LSM Trees:
  - Utilizing compression for storage efficiency.
- Handling Compressed Pages:
  - Addressing challenges due to smaller, non-aligned sizes.
  - Indirection layer for offset and size mapping.
- Read Process and Compression Information:
  - Sequential appending of compressed pages during compaction and flush.
  - Storage of compression information in a separate file segment.
  - Lookup of compressed page offset and size during reads, followed by decompression.

Unordered LSM Storage

Unordered LSM Storage == Store data records in an unordered manner.

They don't require a separate log file, and allow us to reduce cost of writes by storing data records in the order of insertion.

Bitcask

Bitcask is an example of unordered LSM storage.

- It does not use memtables for buffering.
- It stores data records directly in logfiles.
- It uses a data structure called "keydir" that holds references to the latest data records for each key. This is done to make the values searchable.
- During a "write", the data record is appended to the logfile, and the keydir is updated.
- During compaction, contents of the logfiles are merged into a new logfile, and the keydir is updated with new pointers to the relocated data records.

Advantages: Simple. Great point query performance.

Disadvantages: Does not support range queries.

WiscKey

WiskKey keeps the keys sorted in LSM trees, and data records in unordered append-only files called vLogs (value logs).

This helps compaction process to be significantly more efficient, as the keys are typically much smaller than the data records.

Concurrency in LSM Trees

We have the following synchronization points in LSM Trees:

- Memtable switch: After this, all writes go to the new memtable, and the old memtable is available for reads.
- Flush finalization: Replaces the old memtable with a flushed disk-resident table.
- Write-ahead log truncation: Discards the log section holding operations-applied-to-the-flushed-memtable.

Log Stacking

Log stacking == Stacking multiple log-structured systems on top of each other.

This can run into problems like write amplification, fragmentation, and poor performance.

Flash Translation Layer

Log-structured storage is used on SSDs to amortize I/O costs by writing data in batches, which result in smaller number of operations, and hence lesser number of times the garbage collection is triggered.

This also helps SSD wear leveling because of the reduced number of writes.

Filesystem Logging

Having multiple layers of log-structured storage on top of each other could cause writing records that have sizes not aligned with the underlying hardware page size.

This results in fragmentation.

To reduce this interleaving, some database vendors recommend keeping the logs on a separate device.

LLAMA and Mindful Stacking

LLAMA == Latch-free, log-structured, access-method aware storage subsystem

Stacking LLAMA and Bw-Tree demonstrates the benefits of coordination between software layers because of the awareness of the underlying storage subsystem, and hence logical node consolidation.

Conclusion: Stacking, when considered carefully, yields efficiency benefits.

Open-Channel SSDs

Alternative to stacking software layers is to skip them all and directly use the hardware.

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Key Nuances

- B+ trees, not B-trees in engines; I/O O(log_F N) and leaves store records; internals store keys/ptrs.
- ACID Consistency = application invariants (DB ensures A/I/D; constraints help preserve C).
- Dirty write = overwriting uncommitted data. Snapshot isolation: prevents read skew; may allow write skew; lost updates if engine doesn't detect WW conflicts. Checkpointing: retain logs up to BEGIN_CHECKPOINT LSN; fuzzy checkpoints carry DPT/TT; redo starts at - begin.
- LMDB: maintains two meta pages, but snapshots keep many old page versions alive until epoch reclamation. Overflow pages: increase read latency and space amp; defrag/pack to avoid; immutable trees can be fully packed.
- LSM compaction: add explicit leveled vs size-tiered trade-offs and write stall/backpressure behavior.
- SSTables: mention block indexes/fence keys (faster binary search) and partitioned Bloom filters.
- Terminology: WiscKey (not WiskKey); name slot directory; CRC-32C per page